

SQL PROJECT REPORT

BANKING TRANSACTION FRAUD DETECTION

Submitted By	Apurva Khismatrao
Guided By	Mr. Sameer Warsolkar Sir
Course	Data Analyst
Academic Year	2025 – 2026
Technology	MySQL / SQL

Abstract

The Banking Transaction Fraud Detection using SQL project focuses on analyzing banking data to identify potentially fraudulent and suspicious transactions. The project uses a relational database containing customer, account, merchant, transaction, fraud alert, and login activity information. SQL is used to store, retrieve, filter, and analyze the data to identify patterns such as high-value transactions, failed transactions, high-risk merchants, and fraud alerts. Various SQL concepts, including aggregate functions, joins, GROUP BY, HAVING, subqueries, CASE, CTEs, and window functions, are used to generate meaningful results. The project demonstrates how SQL can be applied to banking data analysis and can support the identification and monitoring of potentially suspicious financial activities.

Introduction

Banking transactions generate a large amount of financial data every day. Some transactions may be unusual or suspicious and can indicate possible fraudulent activity.

This project uses **SQL** to analyze banking transaction data and identify suspicious transaction patterns. The project analyzes customers, accounts, merchants, transactions, fraud alerts, and login activity.

SQL queries are used to find high-value transactions, failed transactions, high-risk merchants, fraudulent transactions, customer transaction activity, and other useful patterns.

Objectives

The main objectives of this project are:

1. To analyze banking customer and account data.
2. To analyze transaction amounts and transaction types.

3. To identify failed and suspicious transactions.
4. To identify high-value transactions.
5. To identify transactions made at high-risk merchants.
6. To find transactions marked as fraudulent.
7. To analyze customer transaction activity.
8. To practice SQL JOIN, GROUP BY, HAVING, subqueries, CASE, CTEs, and window functions.
9. To generate useful insights for identifying possible fraudulent activity.

Tools & Technologies

Tool / Technology	Purpose
MySQL Workbench	Create and manage the database, tables, relationships, and execute SQL queries
SQL	Query, analyze, filter, and identify suspicious banking transactions
MySQL	Store and manage the banking fraud detection data

SQL Concepts Used

- SELECT
- WHERE
- AND / OR
- LIKE
- BETWEEN
- Aggregate Functions — COUNT(), SUM(), AVG(), MAX(), MIN()
- JOIN

- GROUP BY
- HAVING
- Subqueries
- Window Functions — RANK(), LAG(), etc.
- CTE
- CASE
- ORDER BY
- LIMIT

Database Design

4.1 Table Description

The database contains 6 main tables:

No.	Table Name	Purpose
1	Customers	Stores customer details such as customer name, email, phone number, city, and registration date.
2	Accounts	Stores bank account information such as account type, balance, and account status.
3	Merchants	Stores merchant information such as merchant name, category, city, and risk level.
4	Transactions	Stores banking transaction details such as transaction date, type, amount, and transaction status.
5	Fraud_Alerts	Stores fraud and suspicious transaction alerts along with the reason and alert date.
6	Login_Activity	Stores customer login information such as login date, login status, device type, and login city.

4.2 Table Structures

4.2.1 Customers Table

DESC Customers;

Field	Type	Null	Key	Default	Extra
customer_id	int	NO	PRI	NULL	auto_increment
customer_name	varchar(100)	NO		NULL	
email	varchar(100)	YES	UNI	NULL	
phone	varchar(15)	YES		NULL	
city	varchar(50)	YES		NULL	
registration_date	date	YES		NULL	

4.2.2 Accounts Table

DESC Accounts;

Field	Type	Null	Key	Default	Extra
account_id	int	NO	PRI	NULL	auto_increment
customer_id	int	NO	MUL	NULL	
account_type	varchar(30)	YES		NULL	
balance	decimal(12,2)	YES		NULL	
account_status	varchar(20)	YES		NULL	

4.2.3 Merchants Table

DESC Merchants;

Field	Type	Null	Key	Default	Extra
merchant_id	int	NO	PRI	NULL	auto_increment
merchant_name	varchar(100)	NO		NULL	
merchant_category	varchar(50)	YES		NULL	
city	varchar(50)	YES		NULL	
risk_level	varchar(20)	YES		NULL	

4.2.4 Transactions Table

DESC Transactions;

Result Grid

 Filter Rows:

Export: 

Wrap Cell Content: 

	Field	Type	Null	Key	Default	Extra
▶	transaction_id	int	NO	PRI	NULL	auto_increment
	account_id	int	NO	MUL	NULL	
	merchant_id	int	YES	MUL	NULL	
	transaction_date	datetime	YES		NULL	
	transaction_type	varchar(30)	YES		NULL	
	amount	decimal(12,2)	YES		NULL	
	transaction_status	varchar(20)	YES		NULL	

4.2.5 Fraud_Alerts Table

DESC Fraud_Alerts;

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	Field	Type	Null	Key	Default	Extra
▶	alert_id	int	NO	PRI	NULL	auto_increment
	transaction_id	int	NO	MUL	NULL	
	fraud_status	varchar(20)	YES		NULL	
	reason	varchar(200)	YES		NULL	
	alert_date	date	YES		NULL	

4.2.6 Login_Activity Table

DESC Login_Activity;

Result Grid

 Filter Rows:

 Export:

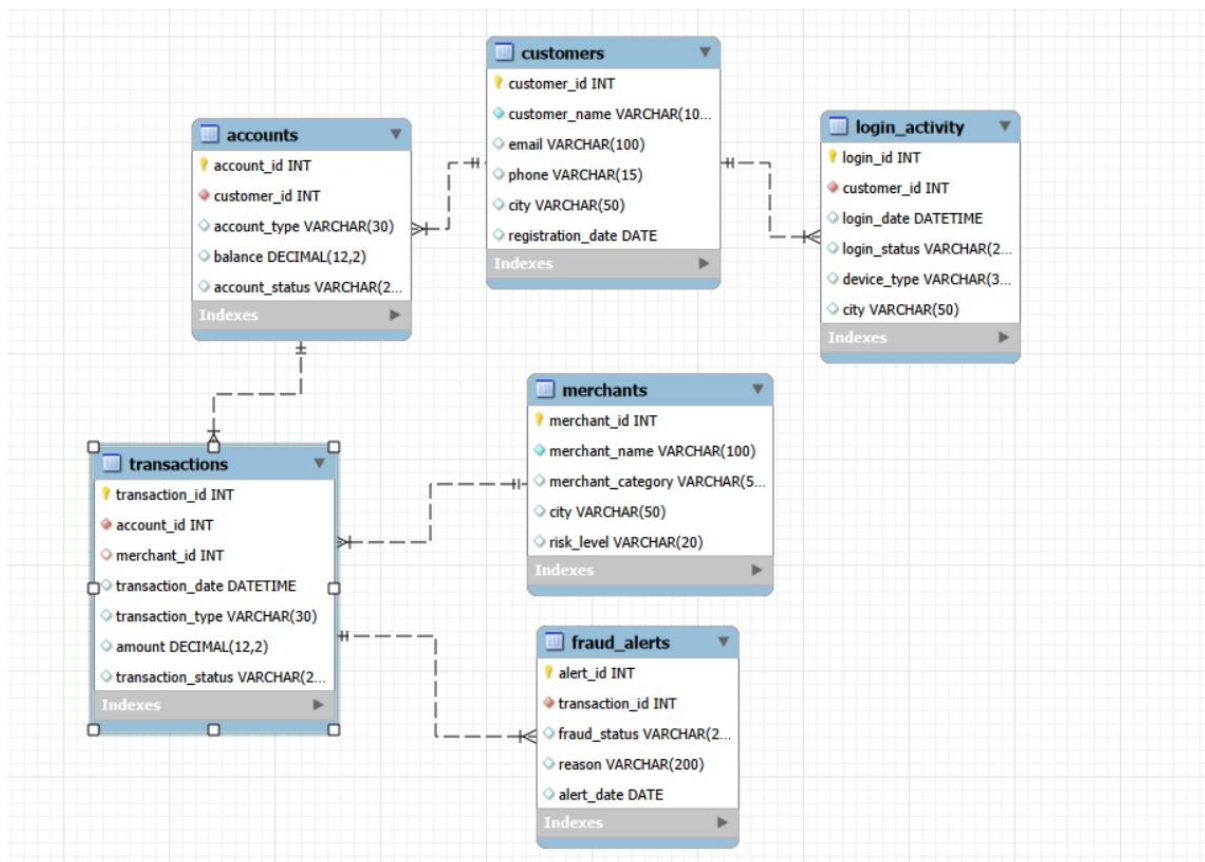
Wrap Cell Content: 

	Field	Type	Null	Key	Default	Extra
▶	login_id	int	NO	PRI	NULL	auto_increment
	customer_id	int	NO	MUL	NULL	
	login_date	datetime	YES		NULL	
	login_status	varchar(20)	YES		NULL	
	device_type	varchar(30)	YES		NULL	
	city	varchar(50)	YES		NULL	

4.3 Main Primary Keys and Foreign Keys

Table	Primary Key	Important Foreign Keys
Customers	customer_id	—
Accounts	account_id	customer_id
Merchants	merchant_id	—
Transactions	transaction_id	account_id, merchant_id
Fraud_Alerts	alert_id	transaction_id
Login_Activity	login_id	customer_id

4.4 Entity Relationship Diagram of Banking Transaction Fraud Detection Database



SQL Queries and Analysis

Database Creation:

```
CREATE DATABASE BankingFraudDetection;
```

```
USE BankingFraudDetection;
```

Table Creation:

Customers

```
CREATE TABLE Customers (  
    customer_id INT PRIMARY KEY AUTO_INCREMENT,  
    customer_name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    phone VARCHAR(15),  
    city VARCHAR(50),  
    registration_date DATE  
);
```

Accounts

```
CREATE TABLE Accounts (  
    account_id INT PRIMARY KEY AUTO_INCREMENT,  
    customer_id INT NOT NULL,
```

```
account_type VARCHAR(30),
balance DECIMAL(12,2),
account_status VARCHAR(20),
FOREIGN KEY (customer_id)
    REFERENCES Customers(customer_id)
);
```

Merchants

```
CREATE TABLE Merchants (
    merchant_id INT PRIMARY KEY AUTO_INCREMENT,
    merchant_name VARCHAR(100) NOT NULL,
    merchant_category VARCHAR(50),
    city VARCHAR(50),
    risk_level VARCHAR(20)
);
```

Transactions

```
CREATE TABLE Transactions (
    transaction_id INT PRIMARY KEY AUTO_INCREMENT,
    account_id INT NOT NULL,
    merchant_id INT,
    transaction_date DATETIME,
    transaction_type VARCHAR(30),
    amount DECIMAL(12,2),
);
```

```
transaction_status VARCHAR(20),  
FOREIGN KEY (account_id)  
REFERENCES Accounts(account_id),  
FOREIGN KEY (merchant_id)  
REFERENCES Merchants(merchant_id)  
);
```

Fraud_Alerts

```
CREATE TABLE Fraud_Alerts (  
    alert_id INT PRIMARY KEY AUTO_INCREMENT,  
    transaction_id INT NOT NULL,  
    fraud_status VARCHAR(20),  
    reason VARCHAR(200),  
    alert_date DATE,  
    FOREIGN KEY (transaction_id)  
        REFERENCES Transactions(transaction_id)  
);
```

Login_Activity

```
CREATE TABLE Login_Activity (  
    login_id INT PRIMARY KEY AUTO_INCREMENT,  
    customer_id INT NOT NULL,  
    login_date DATETIME,  
    login_status VARCHAR(20),  
    device_type VARCHAR(30),
```

```
city VARCHAR(50),  
FOREIGN KEY (customer_id)  
REFERENCES Customers(customer_id)  
);
```

Data Insertion

1. Insert Into Customers Table

```
INSERT INTO Customers  
(customer_name, email, phone, city, registration_date)  
VALUES  
(  
  'Rahul Sharma', 'rahul@gmail.com', '9876543210', 'Mumbai', '2025-01-10'),  
  ('Priya Patil', 'priya@gmail.com', '9876543211', 'Pune', '2025-01-15'),  
  ('Amit Joshi', 'amit@gmail.com', '9876543212', 'Thane', '2025-02-05'),  
  ('Sneha Kulkarni', 'sneha@gmail.com', '9876543213', 'Nashik', '2025-02-20'),  
  ('Rohan Deshmukh', 'rohan@gmail.com', '9876543214', 'Mumbai', '2025-03-01'),  
  ('Neha More', 'neha@gmail.com', '9876543215', 'Pune', '2025-03-15'),  
  ('Akash Jadhav', 'akash@gmail.com', '9876543216', 'Thane', '2025-04-01'),  
  ('Pooja Shinde', 'pooja@gmail.com', '9876543217', 'Mumbai', '2025-04-10'),  
  ('Kunal Pawar', 'kunal@gmail.com', '9876543218', 'Nashik', '2025-05-01'),  
  ('Anjali Sawant', 'anjali@gmail.com', '9876543219', 'Pune', '2025-05-15');
```

```
SELECT * FROM Customers;
```

Result Grid						
Filter Rows:						
Edit:						
Export/Import:						
Wrap Cell Content:						
	customer_id	customer_name	email	phone	city	registration_date
▶	1	Rahul Sharma	rahul@gmail.com	9876543210	Mumbai	2025-01-10
	2	Priya Patil	priya@gmail.com	9876543211	Pune	2025-01-15
	3	Amit Joshi	amit@gmail.com	9876543212	Thane	2025-02-05
	4	Sneha Kulkarni	sneha@gmail.com	9876543213	Nashik	2025-02-20
	5	Rohan Deshmukh	rohan@gmail.com	9876543214	Mumbai	2025-03-01
	6	Neha More	neha@gmail.com	9876543215	Pune	2025-03-15
	7	Akash Jadhav	akash@gmail.com	9876543216	Thane	2025-04-01
	8	Pooja Shinde	pooja@gmail.com	9876543217	Mumbai	2025-04-10
	9	Kunal Pawar	kunal@gmail.com	9876543218	Nashik	2025-05-01
	10	Anjali Sawant	anjali@gmail.com	9876543219	Pune	2025-05-15
•	NULL	NULL	NULL	NULL	NULL	NULL

2. Insert Into Accounts Table

INSERT INTO Accounts

(customer_id, account_type, balance, account_status)

VALUES

(1, 'Savings', 50000.00, 'Active'),
 (2, 'Savings', 75000.00, 'Active'),
 (3, 'Current', 120000.00, 'Active'),
 (4, 'Savings', 45000.00, 'Active'),
 (5, 'Savings', 90000.00, 'Active'),
 (6, 'Current', 150000.00, 'Active'),
 (7, 'Savings', 35000.00, 'Active'),
 (8, 'Savings', 60000.00, 'Active'),
 (9, 'Current', 110000.00, 'Active'),
 (10, 'Savings', 40000.00, 'Blocked');

SELECT * FROM Accounts;

Result Grid					
		Filter Rows:			
		Edit:			
		Export/Import:			
		Wrap Cell Content:			
	account_id	customer_id	account_type	balance	account_status
▶	1	1	Savings	50000.00	Active
	2	2	Savings	75000.00	Active
	3	3	Current	120000.00	Active
	4	4	Savings	45000.00	Active
	5	5	Savings	90000.00	Active
	6	6	Current	150000.00	Active
	7	7	Savings	35000.00	Active
	8	8	Savings	60000.00	Active
	9	9	Current	110000.00	Active
	10	10	Savings	40000.00	Blocked
*	NULL	NULL	NULL	NULL	NULL

3. Insert Into Merchants Table

INSERT INTO Merchants

(merchant_name, merchant_category, city, risk_level)

VALUES

('Amazon India', 'Shopping', 'Mumbai', 'Low'),

('Flipkart', 'Shopping', 'Pune', 'Low'),

('ABC Electronics', 'Electronics', 'Thane', 'Medium'),

('Quick Travel', 'Travel', 'Mumbai', 'Medium'),

('Online Casino', 'Gaming', 'Pune', 'High'),

('XYZ Jewellery', 'Jewellery', 'Mumbai', 'High'),

('Food Corner', 'Food', 'Thane', 'Low'),

('Supermart', 'Grocery', 'Nashik', 'Low'),

('Crypto World', 'Cryptocurrency', 'Mumbai', 'High'),

('Book Store', 'Books', 'Pune', 'Low');

SELECT * FROM Merchants;

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	merchant_id	merchant_name	merchant_category	city	risk_level
▶	1	Amazon India	Shopping	Mumbai	Low
	2	Flipkart	Shopping	Pune	Low
	3	ABC Electronics	Electronics	Thane	Medium
	4	Quick Travel	Travel	Mumbai	Medium
	5	Online Casino	Gaming	Pune	High
	6	XYZ Jewellery	Jewellery	Mumbai	High
	7	Food Corner	Food	Thane	Low
	8	Supermart	Grocery	Nashik	Low
	9	Crypto World	Cryptocurrency	Mumbai	High
	10	Book Store	Books	Pune	Low
✱	NULL	NULL	NULL	NULL	NULL

4. Insert Into Transactions Table

INSERT INTO Transactions

(account_id, merchant_id, transaction_date, transaction_type, amount, transaction_status)

VALUES

(1, 1, '2025-06-01 10:15:00', 'Purchase', 1200.00, 'Success'),
 (2, 2, '2025-06-01 11:20:00', 'Purchase', 2500.00, 'Success'),
 (3, 3, '2025-06-01 12:30:00', 'Purchase', 8500.00, 'Success'),
 (4, 4, '2025-06-01 13:10:00', 'Payment', 3000.00, 'Success'),
 (5, 5, '2025-06-01 14:20:00', 'Purchase', 15000.00, 'Failed'),
 (6, 6, '2025-06-01 15:00:00', 'Purchase', 4500.00, 'Success'),
 (7, 7, '2025-06-01 16:15:00', 'Withdrawal', 5000.00, 'Success'),
 (8, 8, '2025-06-01 17:30:00', 'Purchase', 1800.00, 'Success'),
 (9, 9, '2025-06-01 18:00:00', 'Transfer', 25000.00, 'Success'),
 (10, 10, '2025-06-01 19:10:00', 'Purchase', 7000.00, 'Failed'),

 (1, 3, '2025-06-02 09:30:00', 'Purchase', 22000.00, 'Success'),
 (2, 1, '2025-06-02 12:10:00', 'Purchase', 1800.00, 'Success'),
 (3, 5, '2025-06-02 14:20:00', 'Purchase', 35000.00, 'Success'),

(4, 7, '2025-06-02 16:00:00', 'Payment', 1200.00, 'Success'),
(5, 6, '2025-06-03 10:45:00', 'Purchase', 2800.00, 'Success'),
(6, 8, '2025-06-03 11:30:00', 'Purchase', 3200.00, 'Success'),
(7, 9, '2025-06-03 13:15:00', 'Purchase', 40000.00, 'Success'),
(8, 2, '2025-06-03 15:40:00', 'Purchase', 2100.00, 'Success'),
(9, 6, '2025-06-04 09:50:00', 'Purchase', 50000.00, 'Failed'),
(10, 4, '2025-06-04 17:20:00', 'Payment', 1500.00, 'Success'),

(1, 9, '2025-06-05 08:30:00', 'Purchase', 30000.00, 'Success'),
(2, 10, '2025-06-05 10:00:00', 'Purchase', 900.00, 'Success'),
(3, 6, '2025-06-05 12:30:00', 'Purchase', 27000.00, 'Success'),
(4, 1, '2025-06-05 14:00:00', 'Purchase', 1300.00, 'Success'),
(5, 5, '2025-06-06 09:15:00', 'Purchase', 45000.00, 'Success'),
(6, 3, '2025-06-06 11:45:00', 'Purchase', 7500.00, 'Success'),
(7, 5, '2025-06-06 13:30:00', 'Purchase', 28000.00, 'Failed'),
(8, 7, '2025-06-06 16:20:00', 'Payment', 1100.00, 'Success'),
(9, 9, '2025-06-07 10:10:00', 'Transfer', 60000.00, 'Success'),
(10, 2, '2025-06-07 12:40:00', 'Purchase', 1600.00, 'Success');

SELECT * FROM Transactions;

Result Grid							
Filter Rows:							
Edit:							
Export/Import:							
Wrap Cell Content:							
	transaction_id	account_id	merchant_id	transaction_date	transaction_type	amount	transaction_status
▶	1	1	1	2025-06-01 10:15:00	Purchase	1200.00	Success
	2	2	2	2025-06-01 11:20:00	Purchase	2500.00	Success
	3	3	3	2025-06-01 12:30:00	Purchase	8500.00	Success
	4	4	4	2025-06-01 13:10:00	Payment	3000.00	Success
	5	5	5	2025-06-01 14:20:00	Purchase	15000.00	Failed
	6	6	6	2025-06-01 15:00:00	Purchase	4500.00	Success
	7	7	7	2025-06-01 16:15:00	Withdrawal	5000.00	Success
	8	8	8	2025-06-01 17:30:00	Purchase	1800.00	Success
	9	9	9	2025-06-01 18:00:00	Transfer	25000.00	Success
	10	10	10	2025-06-01 19:10:00	Purchase	7000.00	Failed
	11	1	3	2025-06-02 09:30:00	Purchase	22000.00	Success
	12	2	1	2025-06-02 12:10:00	Purchase	1800.00	Success
	13	3	5	2025-06-02 14:20:00	Purchase	35000.00	Success
	14	4	7	2025-06-02 16:00:00	Payment	1200.00	Success
	15	5	6	2025-06-03 10:45:00	Purchase	2800.00	Success
	16	6	8	2025-06-03 11:30:00	Purchase	3200.00	Success
	17	7	9	2025-06-03 13:15:00	Purchase	40000.00	Success
	18	8	2	2025-06-03 15:40:00	Purchase	2100.00	Success
	19	9	6	2025-06-04 09:50:00	Purchase	50000.00	Failed
	20	10	4	2025-06-04 17:20:00	Payment	1500.00	Success
	21	1	9	2025-06-05 08:30:00	Purchase	30000.00	Success
	22	2	10	2025-06-05 10:00:00	Purchase	900.00	Success
	23	3	6	2025-06-05 12:30:00	Purchase	27000.00	Success
	24	4	1	2025-06-05 14:00:00	Purchase	1300.00	Success
	25	5	5	2025-06-06 09:15:00	Purchase	45000.00	Success

5. Insert Into Fraud_Alerts Table

INSERT INTO Fraud_Alerts

(transaction_id, fraud_status, reason, alert_date)

VALUES

(3, 'Fraud', 'High transaction amount', '2025-06-01'),

(5, 'Suspicious', 'Failed high value transaction', '2025-06-01'),

(10, 'Suspicious', 'Failed transaction', '2025-06-01'),

(11, 'Fraud', 'Unusually high transaction', '2025-06-02'),

(13, 'Fraud', 'High risk merchant transaction', '2025-06-02'),

(19, 'Suspicious', 'High value failed transaction', '2025-06-04'),

(21, 'Fraud', 'High risk merchant transaction', '2025-06-05'),

(23, 'Fraud', 'Large transaction amount', '2025-06-05'),

(25, 'Fraud', 'High risk merchant transaction', '2025-06-06'),
 (27, 'Suspicious', 'Failed high value transaction', '2025-06-06'),
 (29, 'Fraud', 'Very high transfer amount', '2025-06-07');

SELECT * FROM Fraud_Alerts;

Result Grid					
Filter Rows:					
Edit:					
Export/Import:					
Wrap Cell Content:					
	alert_id	transaction_id	fraud_status	reason	alert_date
▶	1	3	Fraud	High transaction amount	2025-06-01
	2	5	Suspicious	Failed high value transaction	2025-06-01
	3	10	Suspicious	Failed transaction	2025-06-01
	4	11	Fraud	Unusually high transaction	2025-06-02
	5	13	Fraud	High risk merchant transaction	2025-06-02
	6	19	Suspicious	High value failed transaction	2025-06-04
	7	21	Fraud	High risk merchant transaction	2025-06-05
	8	23	Fraud	Large transaction amount	2025-06-05
	9	25	Fraud	High risk merchant transaction	2025-06-06
	10	27	Suspicious	Failed high value transaction	2025-06-06
	11	29	Fraud	Very high transfer amount	2025-06-07
✱	NULL	NULL	NULL	NULL	NULL

6. Insert Into Login_Activity Table

INSERT INTO Login_Activity

(customer_id, login_date, login_status, device_type, city)

VALUES

(1, '2025-06-01 08:30:00', 'Success', 'Mobile', 'Mumbai'),
 (2, '2025-06-01 09:00:00', 'Success', 'Laptop', 'Pune'),
 (3, '2025-06-01 09:15:00', 'Success', 'Mobile', 'Thane'),
 (4, '2025-06-01 10:00:00', 'Success', 'Mobile', 'Nashik'),
 (5, '2025-06-01 10:30:00', 'Failed', 'Laptop', 'Mumbai'),
 (6, '2025-06-02 08:45:00', 'Success', 'Mobile', 'Pune'),
 (7, '2025-06-02 09:30:00', 'Success', 'Laptop', 'Thane'),
 (8, '2025-06-02 11:00:00', 'Failed', 'Mobile', 'Mumbai'),
 (9, '2025-06-03 10:15:00', 'Success', 'Laptop', 'Nashik'),

(10, '2025-06-03 12:00:00', 'Failed', 'Mobile', 'Pune'),
 (1, '2025-06-04 09:00:00', 'Success', 'Laptop', 'Mumbai'),
 (3, '2025-06-04 09:45:00', 'Failed', 'Mobile', 'Thane'),
 (5, '2025-06-05 10:30:00', 'Success', 'Mobile', 'Mumbai'),
 (7, '2025-06-05 11:15:00', 'Success', 'Laptop', 'Thane'),
 (9, '2025-06-06 14:00:00', 'Failed', 'Mobile', 'Nashik');

SELECT * FROM Login_Activity;

Result Grid						
Filter Rows:						
Edit:						
Export/Import:						
Wrap Cell Content:						
	login_id	customer_id	login_date	login_status	device_type	city
1	1	1	2025-06-01 08:30:00	Success	Mobile	Mumbai
2	2	2	2025-06-01 09:00:00	Success	Laptop	Pune
3	3	3	2025-06-01 09:15:00	Success	Mobile	Thane
4	4	4	2025-06-01 10:00:00	Success	Mobile	Nashik
5	5	5	2025-06-01 10:30:00	Failed	Laptop	Mumbai
6	6	6	2025-06-02 08:45:00	Success	Mobile	Pune
7	7	7	2025-06-02 09:30:00	Success	Laptop	Thane
8	8	8	2025-06-02 11:00:00	Failed	Mobile	Mumbai
9	9	9	2025-06-03 10:15:00	Success	Laptop	Nashik
10	10	10	2025-06-03 12:00:00	Failed	Mobile	Pune
11	1	1	2025-06-04 09:00:00	Success	Laptop	Mumbai
12	3	3	2025-06-04 09:45:00	Failed	Mobile	Thane
13	5	5	2025-06-05 10:30:00	Success	Mobile	Mumbai
14	7	7	2025-06-05 11:15:00	Success	Laptop	Thane
15	9	9	2025-06-06 14:00:00	Failed	Mobile	Nashik
*	NULL	NULL	NULL	NULL	NULL	NULL

Questions

Q1. Find the total number of customers

COUNT()

SELECT COUNT(customer_id) AS total_customers

FROM Customers;

Result Grid	
Filter Rows:	
Export:	
Wrap Cell Content:	
	total_customers
▶	10

Q2. Find the total transaction amount

SUM()

```
SELECT SUM(amount) AS total_transaction_amount  
FROM Transactions;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	total_transaction_amount			
▶	30			

Q3. Find the average transaction amount

AVG()

```
SELECT AVG(amount) AS average_transaction_amount  
FROM Transactions;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	average_transaction_amount			
▶	14516.666667			

Q4. Find the highest and lowest transaction amount

MAX() + MIN()

```
SELECT  
    MAX(amount) AS highest_transaction,  
    MIN(amount) AS lowest_transaction  
FROM Transactions;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	highest_transaction	lowest_transaction		
▶	60000.00	900.00		

Q5. Find all failed transactions

WHERE

SELECT *

FROM Transactions

WHERE transaction_status = 'Failed';

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	transaction_id	account_id	merchant_id	transaction_date	transaction_type	amount	transaction_status
▶	5	5	5	2025-06-01 14:20:00	Purchase	15000.00	Failed
	10	10	10	2025-06-01 19:10:00	Purchase	7000.00	Failed
	19	9	6	2025-06-04 09:50:00	Purchase	50000.00	Failed
	27	7	5	2025-06-06 13:30:00	Purchase	28000.00	Failed
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q6. Find the number of transactions for each transaction type

GROUP BY + COUNT

SELECT

transaction_type,

COUNT(transaction_id) AS total_transactions

FROM Transactions

GROUP BY transaction_type;

Result Grid




Filter Rows:

Export:



Wrap Cell Content:



	transaction_type	total_transactions
▶	Purchase	23
	Payment	4
	Withdrawal	1
	Transfer	2

Q7. Find the total transaction amount for each transaction type

GROUP BY + SUM()

SELECT

transaction_type,

SUM(amount) AS total_amount

FROM Transactions

GROUP BY transaction_type;

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
transaction_type	total_amount		
Purchase	338700.00		
Payment	6800.00		
Withdrawal	5000.00		
Transfer	85000.00		

Q8. Display customer name, account type and account balance

INNER JOIN

SELECT

c.customer_name,

a.account_type,

a.balance

FROM Customers c

INNER JOIN Accounts a

ON c.customer_id = a.customer_id;

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
customer_name	account_type	balance	
Rahul Sharma	Savings	50000.00	
Priya Patil	Savings	75000.00	
Amit Joshi	Current	120000.00	
Sneha Kulkarni	Savings	45000.00	
Rohan Deshmukh	Savings	90000.00	
Neha More	Current	150000.00	
Akash Jadhav	Savings	35000.00	
Pooja Shinde	Savings	60000.00	
Kunal Pawar	Current	110000.00	
Anjali Sawant	Savings	40000.00	

Q9. Display all customers and their account information

LEFT JOIN

SELECT

```

c.customer_name,
a.account_id,
a.account_type,
a.balance
FROM Customers c
LEFT JOIN Accounts a
ON c.customer_id = a.customer_id;

```

Result Grid				
	customer_name	account_id	account_type	balance
▶	Rahul Sharma	1	Savings	50000.00
	Priya Patil	2	Savings	75000.00
	Amit Joshi	3	Current	120000.00
	Sneha Kulkarni	4	Savings	45000.00
	Rohan Deshmukh	5	Savings	90000.00
	Neha More	6	Current	150000.00
	Akash Jadhav	7	Savings	35000.00
	Pooja Shinde	8	Savings	60000.00
	Kunal Pawar	9	Current	110000.00
	Anjali Sawant	10	Savings	40000.00

Q10. Display all accounts and their customer information

RIGHT JOIN

```

SELECT
c.customer_name,
a.account_id,
a.account_type,
a.balance
FROM Customers c
RIGHT JOIN Accounts a
ON c.customer_id = a.customer_id;

```

Result Grid					Filter Rows:	Export:	Wrap Cell Content:
	customer_name	account_id	account_type	balance			
▶	Rahul Sharma	1	Savings	50000.00			
	Priya Patil	2	Savings	75000.00			
	Amit Joshi	3	Current	120000.00			
	Sneha Kulkarni	4	Savings	45000.00			
	Rohan Deshmukh	5	Savings	90000.00			
	Neha More	6	Current	150000.00			
	Akash Jadhav	7	Savings	35000.00			
	Pooja Shinde	8	Savings	60000.00			
	Kunal Pawar	9	Current	110000.00			
	Anjali Sawant	10	Savings	40000.00			

Q11. Find the total transaction amount for each customer

INNER JOIN + SUM() + GROUP BY

SELECT

c.customer_name,

SUM(t.amount) AS total_transaction_amount

FROM Customers c

INNER JOIN Accounts a

ON c.customer_id = a.customer_id

INNER JOIN Transactions t

ON a.account_id = t.account_id

GROUP BY c.customer_id, c.customer_name;

Result Grid					Filter Rows:	Export:	Wrap Cell Content:
	customer_name	total_transaction_amount					
▶	Rahul Sharma	53200.00					
	Priya Patil	5200.00					
	Amit Joshi	70500.00					
	Sneha Kulkarni	5500.00					
	Rohan Deshmukh	62800.00					
	Neha More	15200.00					
	Akash Jadhav	73000.00					
	Pooja Shinde	5000.00					
	Kunal Pawar	135000.00					
	Anjali Sawant	10100.00					

Q12. Find the number of transactions made by each customer

INNER JOIN + COUNT() + GROUP BY

SELECT

c.customer_name,

COUNT(t.transaction_id) AS total_transactions

FROM Customers c

INNER JOIN Accounts a

ON c.customer_id = a.customer_id

INNER JOIN Transactions t

ON a.account_id = t.account_id

GROUP BY c.customer_id, c.customer_name;

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	customer_name	total_transactions			
▶	Rahul Sharma	3			
	Priya Patil	3			
	Amit Joshi	3			
	Sneha Kulkarni	3			
	Rohan Deshmukh	3			
	Neha More	3			
	Akash Jadhav	3			
	Pooja Shinde	3			
	Kunal Pawar	3			
	Anjali Sawant	3			

Q13. Find customers whose total transaction amount is greater than ₹50,000

GROUP BY + HAVING

SELECT

c.customer_name,

SUM(t.amount) AS total_transaction_amount

FROM Customers c

INNER JOIN Accounts a

```

ON c.customer_id = a.customer_id
INNER JOIN Transactions t
ON a.account_id = t.account_id
GROUP BY c.customer_id, c.customer_name
HAVING SUM(t.amount) > 50000;

```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	customer_name	total_transaction_amount			
▶	Rahul Sharma	53200.00			
	Amit Joshi	70500.00			
	Rohan Deshmukh	62800.00			
	Akash Jadhav	73000.00			
	Kunal Pawar	135000.00			

Q14. Find all transactions marked as Fraud

JOIN + WHERE

```

SELECT
    t.transaction_id,
    t.account_id,
    t.amount,
    f.fraud_status,
    f.reason
FROM Transactions t
INNER JOIN Fraud_Alerts f
ON t.transaction_id = f.transaction_id
WHERE f.fraud_status = 'Fraud';

```

Result Grid

 Filter Rows:

 Export:

 Wrap Cell Content:

	transaction_id	account_id	amount	fraud_status	reason
▶	3	3	8500.00	Fraud	High transaction amount
	11	1	22000.00	Fraud	Unusually high transaction
	13	3	35000.00	Fraud	High risk merchant transaction
	21	1	30000.00	Fraud	High risk merchant transaction
	23	3	27000.00	Fraud	Large transaction amount
	25	5	45000.00	Fraud	High risk merchant transaction
	29	9	60000.00	Fraud	Very high transfer amount

Q15. Find transactions made at high-risk merchants

JOIN + WHERE

SELECT

t.transaction_id,
m.merchant_name,
m.risk_level,
t.amount,
t.transaction_status

FROM Transactions t

INNER JOIN Merchants m

ON t.merchant_id = m.merchant_id

WHERE m.risk_level = 'High';

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	transaction_id	merchant_name	risk_level	amount	transaction_status
▶	5	Online Casino	High	15000.00	Failed
	13	Online Casino	High	35000.00	Success
	25	Online Casino	High	45000.00	Success
	27	Online Casino	High	28000.00	Failed
	6	XYZ Jewellery	High	4500.00	Success
	15	XYZ Jewellery	High	2800.00	Success
	19	XYZ Jewellery	High	50000.00	Failed
	23	XYZ Jewellery	High	27000.00	Success
	9	Crypto World	High	25000.00	Success
	17	Crypto World	High	40000.00	Success
	21	Crypto World	High	30000.00	Success
	29	Crypto World	High	60000.00	Success

Q16. Find failed transactions where amount is greater than ₹10,000

WHERE + AND

SELECT

transaction_id,

account_id,

amount,

transaction_status

FROM Transactions

WHERE transaction_status = 'Failed'

AND amount > 10000;

Result Grid	Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
transaction_id	account_id	amount	transaction_status	
5	5	15000.00	Failed	
19	9	50000.00	Failed	
27	7	28000.00	Failed	
NULL	NULL	NULL	NULL	

Q17. Find transactions whose amount is greater than the average transaction amount

Subquery + AVG()

SELECT

transaction_id,

account_id,

amount

FROM Transactions

WHERE amount > (

SELECT AVG(amount)

FROM Transactions

);

Result Grid			
Filter Rows:		Edit:	
Export/Import:		Wrap Cell Content:	
transaction_id	account_id	amount	
5	5	15000.00	
9	9	25000.00	
11	1	22000.00	
13	3	35000.00	
17	7	40000.00	
19	9	50000.00	
21	1	30000.00	
23	3	27000.00	
25	5	45000.00	
27	7	28000.00	
29	9	60000.00	
HULL	HULL	HULL	

Q18. Categorize transactions as Low, Medium or High

CASE

SELECT

transaction_id,

amount,

CASE

WHEN amount < 5000 THEN 'Low'

WHEN amount BETWEEN 5000 AND 20000 THEN 'Medium'

ELSE 'High'

END AS transaction_category

FROM Transactions;

Result Grid			
		Filter Rows:	
		Export:	Wrap Cell Content:
	transaction_id	amount	transaction_category
▶	1	1200.00	Low
	2	2500.00	Low
	3	8500.00	Medium
	4	3000.00	Low
	5	15000.00	Medium
	6	4500.00	Low
	7	5000.00	Medium
	8	1800.00	Low
	9	25000.00	High
	10	7000.00	Medium
	11	22000.00	High
	12	1800.00	Low
	13	35000.00	High
	14	1200.00	Low
	15	2800.00	Low
	16	3200.00	Low
	17	40000.00	High
	18	2100.00	Low
	19	50000.00	High
	20	1500.00	Low
	21	30000.00	High
	22	900.00	Low
	23	27000.00	High
	24	1300.00	Low
	25	45000.00	High
	26	7500.00	Medium
	27	28000.00	High

Q19. Calculate total transaction amount for each customer using CTE

CTE + SUM()

WITH CustomerTransactions AS (

SELECT

c.customer_name,

SUM(t.amount) AS total_amount

FROM Customers c

INNER JOIN Accounts a

ON c.customer_id = a.customer_id

INNER JOIN Transactions t

ON a.account_id = t.account_id

```

GROUP BY c.customer_id, c.customer_name
)
SELECT *
FROM CustomerTransactions;

```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	customer_name	total_amount			
▶	Rahul Sharma	53200.00			
	Priya Patil	5200.00			
	Amit Joshi	70500.00			
	Sneha Kulkarni	5500.00			
	Rohan Deshmukh	62800.00			
	Neha More	15200.00			
	Akash Jadhav	73000.00	15200.00		
	Pooja Shinde	5000.00			
	Kunal Pawar	135000.00			
	Anjali Sawant	10100.00			

Q20. Rank customers based on total transaction amount

RANK() + Window Function

```

WITH CustomerTotals AS (
    SELECT
        c.customer_name,
        SUM(t.amount) AS total_amount
    FROM Customers c
    INNER JOIN Accounts a
        ON c.customer_id = a.customer_id
    INNER JOIN Transactions t
        ON a.account_id = t.account_id
    GROUP BY c.customer_id, c.customer_name
)
SELECT
    customer_name,

```

```
total_amount,
RANK() OVER (ORDER BY total_amount DESC) AS customer_rank
FROM CustomerTotals;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
	customer_name	total_amount	customer_rank
▶	Kunal Pawar	135000.00	1
	Akash Jadhav	73000.00	2
	Amit Joshi	70500.00	3
	Rohan Deshmukh	62800.00	4
	Rahul Sharma	53200.00	5
	Neha More	15200.00	6
	Anjali Sawant	10100.00	7
	Sneha Kulkarni	5500.00	8
	Priya Patil	5200.00	9
	Pooja Shinde	5000.00	10

Conclusion

The Banking Transaction Fraud Detection using SQL project successfully demonstrates how SQL can be used to store, manage, and analyze banking transaction data. The project integrates customer, account, merchant, transaction, fraud alert, and login activity information into a structured relational database.

Using SQL queries, we analyzed transaction amounts, transaction types, customer activity, failed transactions, high-risk merchants, and fraudulent transactions. Different SQL concepts such as aggregate functions, WHERE, GROUP BY, HAVING, INNER JOIN, LEFT JOIN, RIGHT JOIN, subqueries, CASE, CTE, and window functions were used to extract meaningful information from the database.

Overall, this project provides a practical understanding of how SQL can support fraud monitoring and transaction analysis and help identify potentially suspicious banking activities.